

07 — DATABASE DESIGN DOCUMENT

SENTINEL FUSION

AI-Powered Digital Intelligence Fusion Platform
for Digital Forensics & Investigations

Phase 3 — Design & Planning

Database architecture, logical schema, data flow, entity relationships, AI processing flow, security and storage design

1. Introduction

This Database Design Document defines the logical organization, storage, relationships and management of data for Sentinel Fusion. The design follows the supplied product requirements and technology stack: PostgreSQL for transactional data, Neo4j for relationship intelligence, Qdrant for embeddings, Redis for queues/caching, OpenSearch for search, and object storage for large multimedia evidence.

2. Database Design Principles

- Evidence integrity: original evidence is preserved in object storage and referenced by cryptographic hash.
- Separation of concerns: relational, graph, vector, search and binary workloads use specialized stores.
- Human oversight: AI findings are stored as explainable, reviewable records rather than autonomous conclusions.
- Auditability: user actions and evidence operations are recorded in audit logs.

3. High-Level Database Architecture

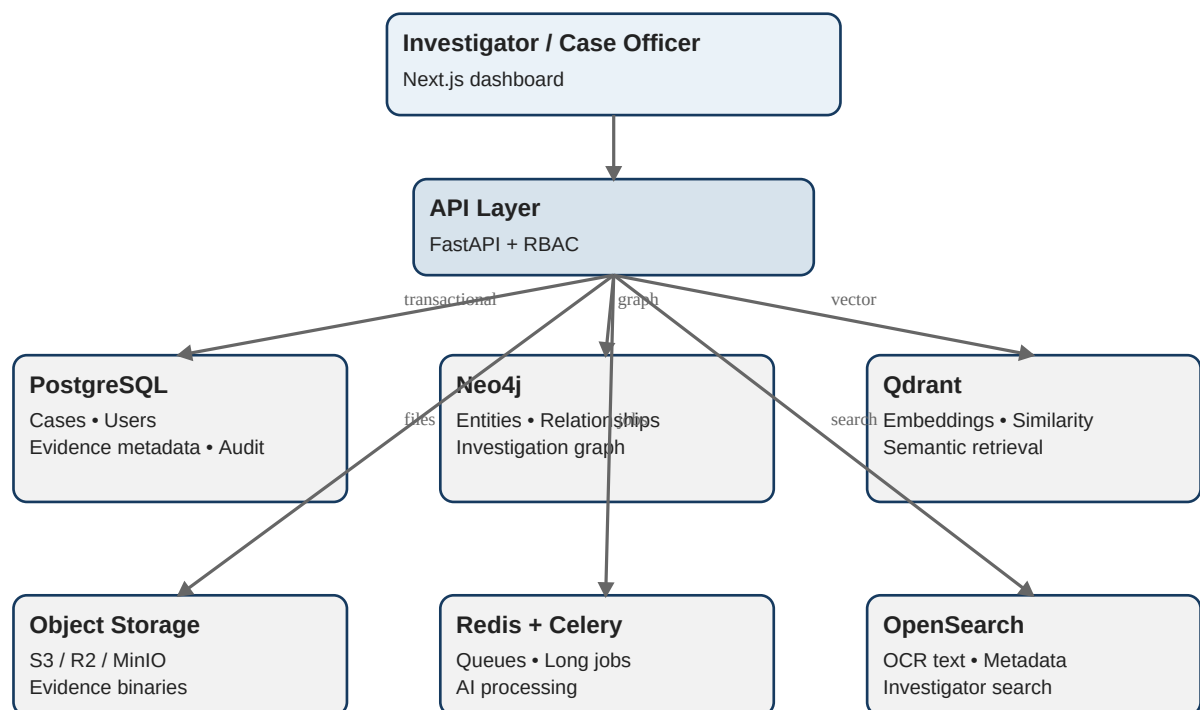


Figure 1. Sentinel Fusion hybrid database architecture.

4. Entity Relationship Diagram

The core relational model centers on cases and evidence. Users create or work on cases; evidence belongs to cases; metadata and AI findings are associated with evidence; timelines and reports are associated with cases; audit logs preserve user activity.

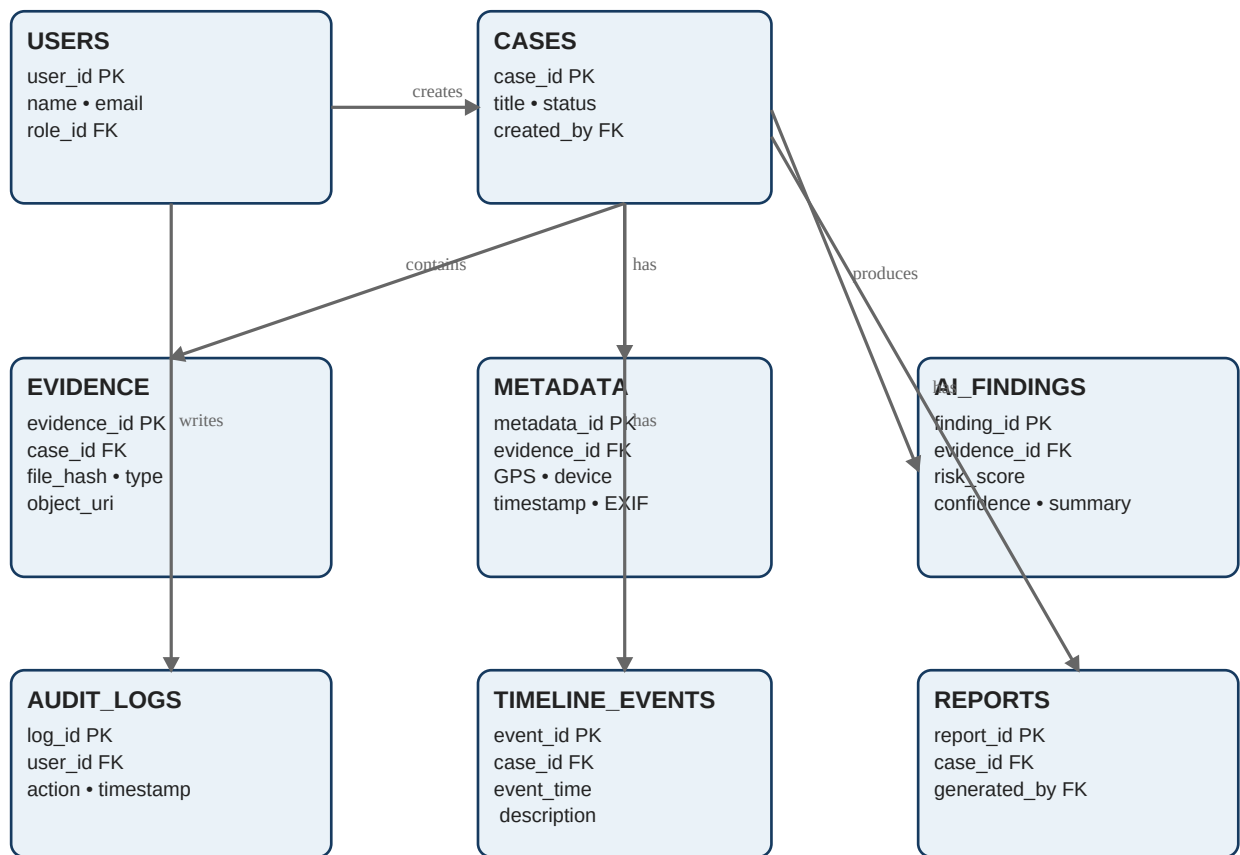


Figure 2. Core relational entity relationship model.

5. Core Relational Entities

Entity	Purpose	Primary Key
USERS	Investigator and authorized user accounts	user_id
ROLES	Role definitions for RBAC	role_id
CASES	Investigation/case records	case_id
EVIDENCE	Evidence inventory and integrity metadata	evidence_id
METADATA	Extracted technical and contextual metadata	metadata_id
AI_FINDINGS	AI-generated findings, confidence and risk	finding_id
TIMELINE_EVENTS	Chronological investigation events	event_id
REPORTS	Generated investigation reports	report_id
AUDIT_LOGS	Immutable-style audit trail of actions	log_id

6. Evidence Ingestion & Database Flowchart

Sentinel Fusion treats evidence ingestion as a controlled pipeline. A file is hashed before downstream processing, stored outside the relational database, registered in PostgreSQL, processed by specialist AI/forensic components, and then indexed and correlated.

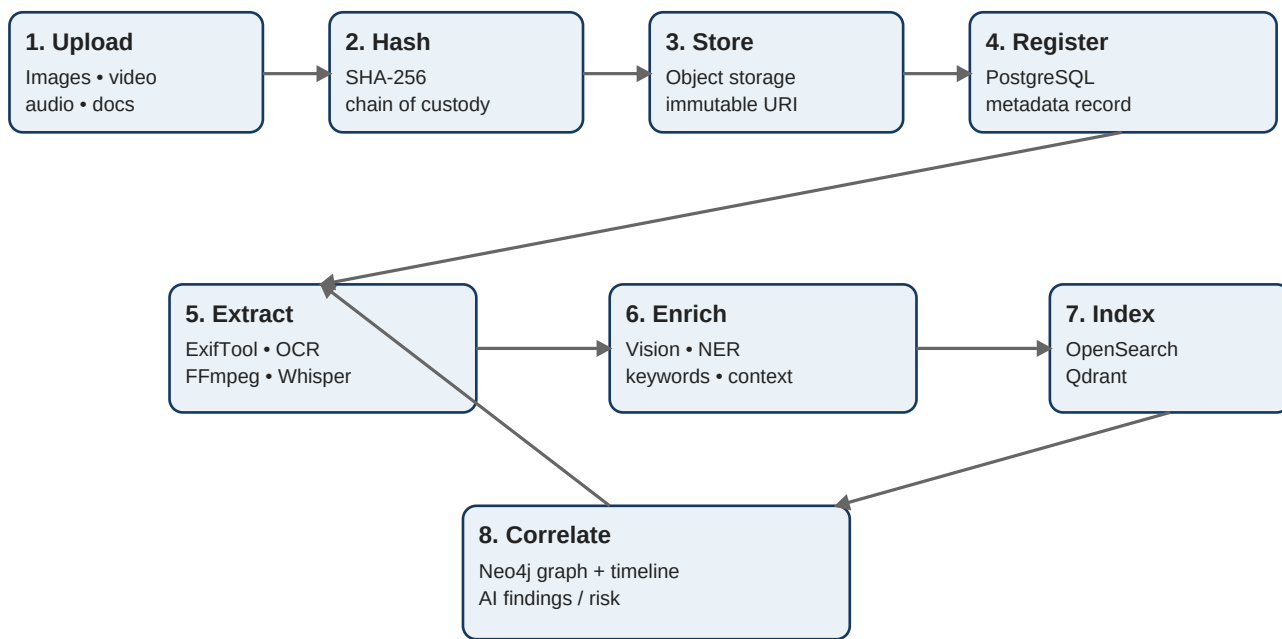


Figure 3. Evidence ingestion, extraction, indexing and correlation flow.

7. Evidence Storage Model

Large evidence binaries such as images, videos, audio recordings, documents and archives should not be stored directly inside PostgreSQL. Instead, the binary object is stored in S3, Cloudflare R2 or MinIO, while PostgreSQL stores its object URI, SHA-256 hash, type, ownership and case association. This keeps transactional queries efficient while preserving a clear evidence reference.

8. AI Processing & Human Review Flow

The AI layer combines extraction, enrichment and reasoning components. The resulting risk score, graph relationships, timeline events and summaries are persisted as reviewable outputs. The investigator remains responsible for validating or rejecting findings.

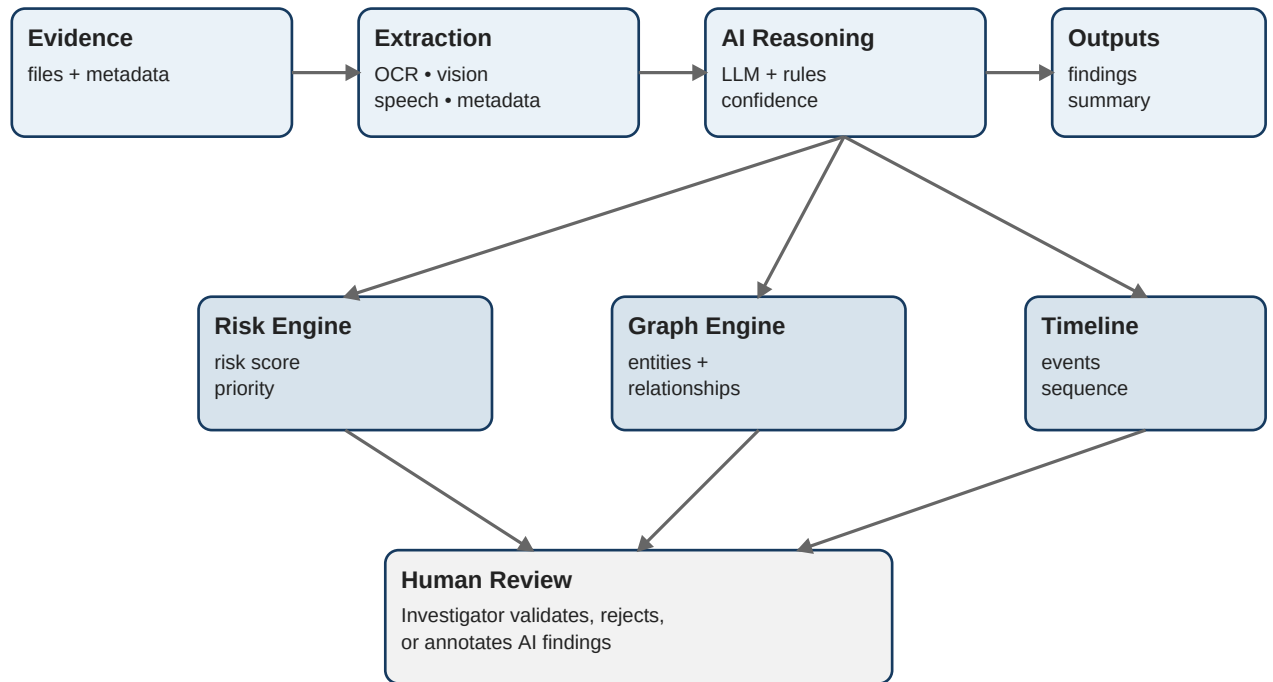


Figure 4. AI processing pipeline with human-in-the-loop review.

9. Case Investigation Data Flow

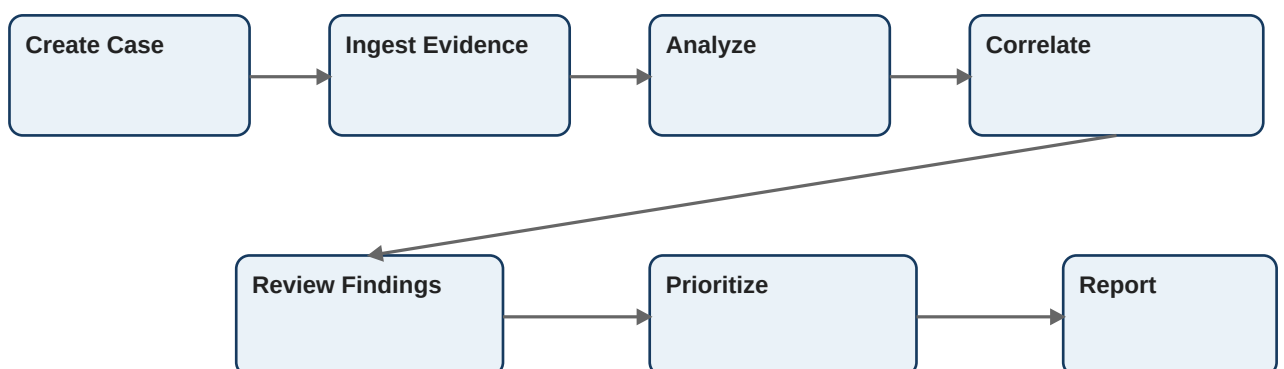


Figure 5. End-to-end case workflow from evidence ingestion to reporting.

10. Relational Schema

The following logical schema is derived from the supplied PRD and technology stack.

Table	Key fields	Relationships / notes
USERS	user_id PK; name; email; role_id FK	Many users can share a role.
ROLES	role_id PK; role_name	Referenced by USERS.
CASES	case_id PK; title; status; created_by FK	One case contains many evidence items, events and reports.
EVIDENCE	evidence_id PK; case_id FK; file_hash; object_uri; type; uploaded_by FK	Core evidence inventory.
METADATA	metadata_id PK; evidence_id FK; gps; device; timestamp; exif_json	Technical metadata extracted from evidence.
AI_FINDINGS	finding_id PK; evidence_id FK; risk_score; summary; confidence	AI-generated, explainable findings.
TIMELINE_EVENTS	event_id PK; case_id FK; event_time; description	Chronological reconstruction.
REPORTS	report_id PK; case_id FK; generated_by FK	Investigation report outputs.
AUDIT_LOGS	log_id PK; user_id FK; action; timestamp	Tracks user and evidence operations.

11. Graph Database Model — Neo4j

Neo4j complements PostgreSQL by representing connected intelligence. Recommended nodes include Person, Device, Account, Phone, Email, Location, Evidence and Case. Recommended relationships include OWNS, CONTACTED, LOCATED_AT, APPEARS_IN, RELATED_TO and UPLOADED_BY. This model supports network visualization and hidden relationship discovery without forcing graph workloads into relational joins.

12. Data Dictionary

Field	Type (logical)	Description	Constraints
user_id	UUID / BIGINT	Unique user identifier	PK, NOT NULL
role_id	UUID / BIGINT	RBAC role identifier	FK → ROLES
case_id	UUID / BIGINT	Unique investigation identifier	PK, NOT NULL
evidence_id	UUID / BIGINT	Unique evidence identifier	PK, NOT NULL
file_hash	CHAR(64)	SHA-256 evidence hash	UNIQUE, NOT NULL
object_uri	TEXT	Object storage reference	NOT NULL
type	VARCHAR	Evidence media/file type	NOT NULL
gps	JSON / spatial	Extracted location information	Optional
timestamp	TIMESTAMP	Evidence or event timestamp	Indexed
risk_score	DECIMAL	AI risk prioritization score	Range validated
confidence	DECIMAL	AI confidence score	Range validated
event_time	TIMESTAMP	Timeline event time	Indexed
action	VARCHAR	Audit action name	NOT NULL

13. Normalization

First Normal Form (1NF): each relational field represents a single logical value and repeating groups are separated.

Second Normal Form (2NF): non-key attributes depend on the complete primary key.

Third Normal Form (3NF): non-key attributes do not depend on other non-key attributes. Role information is separated into ROLES rather than duplicated in USERS; evidence metadata is separated from EVIDENCE.

14. Indexing Strategy

Recommended indexes include case_id, evidence_id, SHA-256 file_hash, user email, event timestamps and risk_score. OpenSearch handles large-scale OCR/metadata text search, while Qdrant handles embedding similarity and semantic retrieval.

15. Security & Evidence Integrity

The supplied stack specifies TLS 1.3, AES-256 encryption at rest, signed URLs, RBAC, MFA, immutable audit logs and evidence hash verification. Database access should follow least privilege. Evidence records should retain their SHA-256 hash so later verification can detect unexpected changes.

16. Backup & Recovery

PostgreSQL should use scheduled backups and point-in-time recovery where available. Neo4j should have database snapshots. Object storage should use versioning/retention controls appropriate for evidentiary material. Recovery procedures should be tested rather than only documented.

17. Representative SQL Schema

```
CREATE TABLE roles (  
    role_id UUID PRIMARY KEY,  
    role_name VARCHAR(50) UNIQUE NOT NULL  
);  
  
CREATE TABLE cases (  
    case_id UUID PRIMARY KEY,  
    title TEXT NOT NULL,  
    status VARCHAR(30) NOT NULL,  
    created_by UUID NOT NULL REFERENCES users(user_id)  
);  
  
CREATE TABLE evidence (  
    evidence_id UUID PRIMARY KEY,  
    case_id UUID NOT NULL REFERENCES cases(case_id),  
    file_hash CHAR(64) UNIQUE NOT NULL,  
    object_uri TEXT NOT NULL,  
    type VARCHAR(50) NOT NULL,  
    uploaded_by UUID REFERENCES users(user_id)  
);  
  
CREATE INDEX idx_evidence_case ON evidence(case_id);  
CREATE INDEX idx_evidence_hash ON evidence(file_hash);
```

Note: this is a representative logical schema. Final implementation types and constraints should be aligned with the application code and deployment environment.

18. Conclusion

The Sentinel Fusion database design uses a polyglot persistence model appropriate for an AI-native forensic intelligence platform. PostgreSQL provides reliable transactional records; Neo4j provides connected investigation intelligence; Qdrant provides semantic retrieval; OpenSearch provides evidence search; Redis supports asynchronous processing; and object storage preserves large evidence files. Together, these components provide a scalable foundation while retaining auditability, evidence integrity and human oversight.

Prepared for Phase 3 — Design & Planning